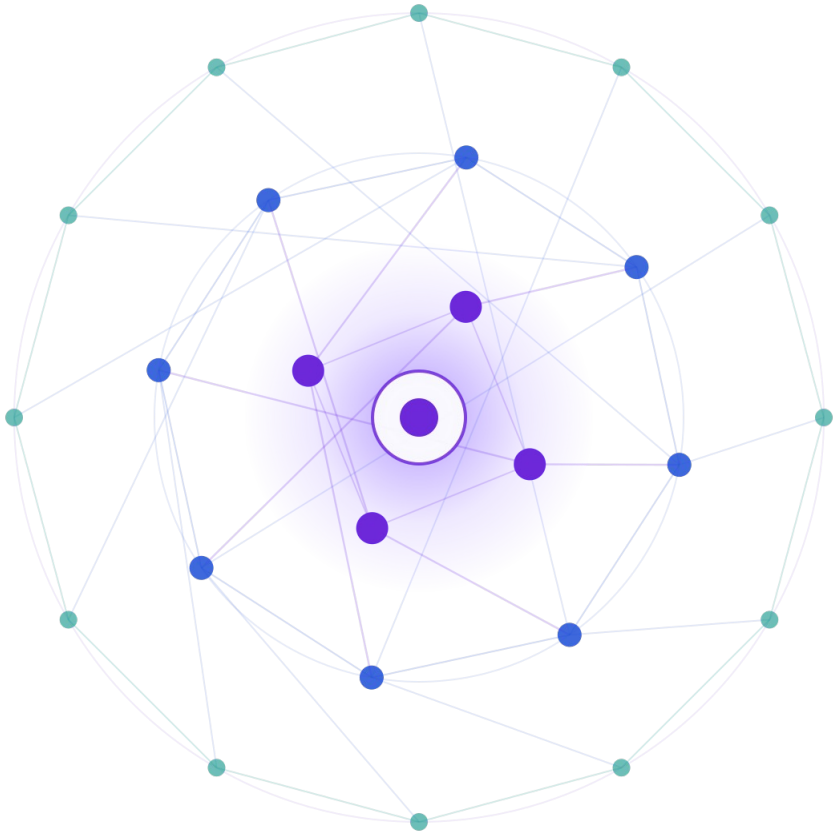
SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme 01 - CodeDoctor
- Team - Doctors
- College - VITV
- Member 1 / Email:
- Member 2 / Email:
- Member 3 / Email:
- Member 4 / Email:



Submission GitHub: CodeDoctor repository

Theme

- **Theme 01: code retrieval** Rank relevant code snippets for a natural-language query.
- **Our focus** Pre/post-processing around a pinned embedding model, with reusable indexes and behavioral checks.
- **P0** Semantic retrieval followed by execution verification of worked examples.
- **P1 and Bonus** Rebuild after changes and retrieve useful variants across repository versions.

Existing Solutions & Gaps

- **Lexical search** Useful for names and identifiers, but wording can differ from implementation.
- **Dense retrieval** Matches intent, but near-identical programs can have different behavior.
- **Full re-indexing** Repeatedly embeds unchanged code after small edits.
- **Our response** Reuse representations before retrieval, check behavior afterward, and preserve version provenance.

Our Solutions & Architecture Diagram

Our pre/post-processing contribution

PREPROCESS

Exact corpus text
Structure-aware chunks*
Recipe-keyed vectors
Incremental reuse*

RETRIEVE

1.7B semantic encoder
CPU query inference
Cosine candidates
Hybrid routing in demo*

POSTPROCESS

Parse worked examples
Execute top candidates
Verify before grouping*
Return ranked evidence

* Repository workbench features. P0 uses corpus vectors and execution reranking.
Shared encoder recipe and execution settings connect both paths.

Demo & Product Walkthrough

- **1. Index a repository** Build structure-aware chunks and cache document embeddings.
- **2. Retrieve code** Search in natural language and inspect source spans and verification evidence.
- **3. Edit and rebuild** Re-encode changed chunks while reusing unchanged vectors.
- **4. Compare versions** Retrieve across versions and inspect the selected variant and provenance.
- **Demo video** Recording link:

Tools and tech stack used

- **Encoder** F2LLM-v2-1.7B, pinned weights, published prompt and normalized last-token pooling.
- **Preprocessing** Python, tree-sitter, SQLite embedding cache and Git version identities.
- **Postprocessing** Example parser, resource-limited Python execution and shared verification settings.
- **Application and evaluation** FastAPI workbench, official MTEB AppsRetrieval, Kaggle stage runners and source hashes.
- **Optional tier** 0.6B interactive mode is separate from the 1.7B accuracy submission.

Impact & Use case

- **Repository maintenance** Find relevant functions and inspect exact file and line references.
- **Regression investigation** Compare similar variants using worked examples rather than semantic similarity alone.
- **Changing codebases** Reuse unchanged embeddings across incremental rebuilds and file moves.
- **Judge-facing evidence** P0 accuracy, P1 rebuild cost and cross-version diagnostics reported separately.

Innovation highlights, results and limitations

- **Architectural contribution** Novel pre/post-processing architecture around open-weights F2LLM-v2-1.7B: reusable embeddings before retrieval, behavioral execution checks after retrieval.

- **Retrieval quality (P0)** Historical AppsRetrieval benchmark: NDCG@10 0.96558, MRR@10 0.95857. Staged CPU retrieval pipeline minimizes GPU reliance.

- **Rebuild & versions (P1 / Bonus)** P1 rebuild reuses unchanged embeddings, cutting update time from ~92s to ~1.4s. Bonus correctly selects working variants across 12 lineages.

- **Honest limitations** Passing visible examples does not prove full correctness. Subprocess sandbox is resource-capped but not isolated. Plausible APPS training exposure is disclosed.

What’s next

- **Unseen Python evaluation** Demonstrate retrieval and verification on the judges’ unreleased data.
- **Broader validation** Expand realistic version histories and independently check example extraction.
- **Efficiency** Profile CPU bottlenecks before any further accelerator experiment.
- **Reproducibility** Preserve run evidence, configuration history and negative experimental findings.

Brownie points slide(differentiation)

- **Behavior-aware ranking** Semantic similarity proposes candidates; executable examples supply additional evidence.
- **Version-aware selection** Verify variants before grouping, remove unchanged duplicates and retain provenance.
- **Change-aware indexing** Content and encoder recipe determine reuse, independent of path and line changes.
- **Auditable evaluation** Pinned source and model, official evaluator, raw logs and explicit exposure disclosures.

Checklist - Submission materials

- **Working prototype** Full source code on GitHub main branch with reproducible local & Docker setup.
- **README & Requirements** Detailed README, requirement.txt, and submission protocol.
- **Presentation & PDF** Completed 12-slide deck (VITV_Doctors_Submission.pptx) and matching PDF.
- **AI Disclosure & Tag** LangAI3.0_AI_Disclosure.docx included; release tag: PRISM_GENAI_HACKATHON_Y2026.
- **Demo video** Video walkthrough (maximum 5 minutes): [Google Drive / YouTube Link].

Thank you

CodeDoctor / VITV / Doctors

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India